المدرسة الوطنية للعلوم التطبيقية

Ecole Nationale des Sciences Appliquées

Mini-rapport — Semaine 1 (S1)

Cadrage et théorie

Projet 1 — Solveur de Nash $m \times n$
et duopoles (Cournot, Bertrand)

Groupe 1 : ACHTOUK Fatine, ADIL Youness, BELHIMER Ikram, BOULHAIL Hiba

Module : Théorie des Jeux et Systèmes Multi-Agents — ENSA Safi

Professeur : Abdelilah JRAIFI — Année universitaire 2026-2027

12 novembre 2026

1. Objectif de la semaine

La semaine 1 est consacrée au **cadrage** du projet et à la **théorie** : poser les fondations avant d'écrire le code. L'objectif est de disposer d'un dépôt, d'un environnement de travail, d'une fiche de lecture théorique et d'un plan de travail (Gantt) validés par l'équipe.

Table 1 – Bilan des attendus de la semaine 1

Attendu (guide)	**Réalisation**	**État**
Dépôt GitHub créé	Dépôt du Groupe 1, branche `main`	✓
Ollama installé + `ollama pull llama3.1`	`llama3.1` (8B, *tools*) + `llama3.2:3b`, serveur sur `127.0.0.1:11434`	✓
Fiche de lecture (Phase 0) rendue	`docs/fiche_lecture.tex` → PDF	✓
Fonctions du cours à étendre identifiées	`nash_mixte_2x2` → `enumeration_supports`, `est_nash`, `cournot`, `cournot_grille`	✓
Gantt prévisionnel déposé	`docs/gantt/` : Mermaid + CSV + PNG	✓

2. Travail réalisé, par rôle

Conformément à la règle n° 4, chaque membre a un rôle propre et au moins un commit signé de son prénom.

- **Théorie — Fatine (chef de projet).** Formules de Nash, d'indifférence sur les supports, de Cournot et de Bertrand rédigées en LaTeX et expliquées « avec ses mots » ; hypothèses et limites listées ; conventions A/B fixées avec les Groupes 2 et 6.
- **Quant / Python — Ikram.** `src/jeux.py` : `est_nash`, `enumeration_supports`, `cournot`, `cournot_grille`, `bertrand` et les jeux de référence du cours (dilemme, bataille des sexes, pierre-feuille-ciseaux).
- **Développeur agent — Youness.** `src/agent_tj_sma.py` : les deux nouveaux outils exposés au LLM, `resoudre_bimatrice(A, B)` et `cournot(a, b, c, n_firmes)`, avec plafonds et sortie JSON.
- **Testeur / Expérimentateur — Hiba.** `tests/` : validation sur les trois jeux du cours, test hors ligne des outils (`.invoke`), ébauche de `src/experiences.py`.

3. Rappels théoriques (formules clés)

Équilibre de Nash. Un profil (x, y) est un équilibre de Nash si aucune déviation *pure* n'est profitable :

$$(Ay)_i \leq x^\top Ay \quad \forall i, \qquad (x^\top B)_j \leq x^\top By \quad \forall j.$$

Indifférence sur les supports. Si I (resp. J) est le support du joueur 1 (resp. 2), avec $|I| = |J| = k$, alors sur le support les gains purs sont égaux :

$$(Ay)_i = v_1 \ \ (i \in I), \qquad (x^\top B)_j = v_2 \ \ (j \in J),$$

d'où un système linéaire que l'on résout pour chaque couple de supports — c'est le principe de l'**énumération des supports**.

Cournot. Avec $P(Q) = a - bQ$ et coût marginal c, la meilleure réponse est $q_1 = (a - c - b\,q_2)/(2b)$; à l'équilibre symétrique à n firmes,

$$q^* = \frac{a-c}{(n+1)\,b}, \qquad \pi^* = \frac{(a-c)^2}{b\,(n+1)^2}.$$

Pour le duopole de référence ($a = 100$, $b = 1$, $c = 10$) : $q^* = 30$, prix 40, profit 900 par firme ; le cartel donnerait $q = 22{,}5$ et un profit 1012,5.

Bertrand. Concurrence en prix sur biens homogènes : l'équilibre est $p_1 = p_2 = c$ et le profit est **nul** — le *paradoxe de Bertrand.*

4. Livrables et vérifications

- `python -m pytest -q` : les tests passent (fonctions pures + outils sans LLM, aucun appel réseau).
- `python src/jeux.py` : affiche les équilibres des trois jeux du cours et Cournot ($q^* = 30$, profit 900).
- `ollama list` : `llama3.1` (8B, *tools*) disponible pour l'agent.
- `docs/fiche_lecture.pdf` (Phase 0) et `docs/gantt/` (Gantt prévisionnel) déposés dans le dépôt.

5. Difficultés rencontrées et solutions

- **Python 3.15 vs `zstandard`.** `langchain-core` → `langsmith` → `zstandard`, qui n'a pas de wheel pour Python 3.15 ; `pip` tentait de le compiler et échouait (Visual C++ requis). *Solution* : environnement dédié `.venv` sur **Python 3.12**, où tous les paquets s'installent en binaire (documenté dans `requirements.txt`).
- **Matrices passées en chaîne.** `llama3.1` envoie parfois $A =$ `"[[3, 0], [5, 1]]"` ; le validateur pydantic de LangChain rejetait la chaîne. *Solution* : type `Matrice = Union[list, str]` dans `agent_tj_sma.py` (décodage `json.loads`, jamais `eval`), avec un test associé.
- **`ollama` absent du `PATH`.** Lancer l'application Ollama (serveur sur `127.0.0.1:11434`) ou appeler le chemin complet `%LOCALAPPDATA%\Programs\Ollama\ollama.exe`.

6. Écarts prévu / réalisé

Aucun écart bloquant. L'installation de LangChain, initialement problématique, a été résolue en travaillant sur Python 3.12 : l'agent LangChain réel est donc opérationnel *dès S1*, soit une **avance** sur le planning prévisionnel (prévu en S3–S4). Le reste est conforme au Gantt prévisionnel.

7. Gantt prévisionnel

8. Plan de la semaine suivante (S2)

- Exécuter les fonctions dans `notebooks/01_theorie_prototype.ipynb` et reproduire les résultats du cours.
- Rédiger la section « formules expliquées » du rapport final.
- Lancer les 20 jeux aléatoires (`src/experiences.py`) et le banc d'essai **5 énoncés** × **2 modèles** Ollama.

9. État du dépôt Git (commits de la semaine)

Chaque membre a poussé au moins un commit signé de son prénom entre crochets :

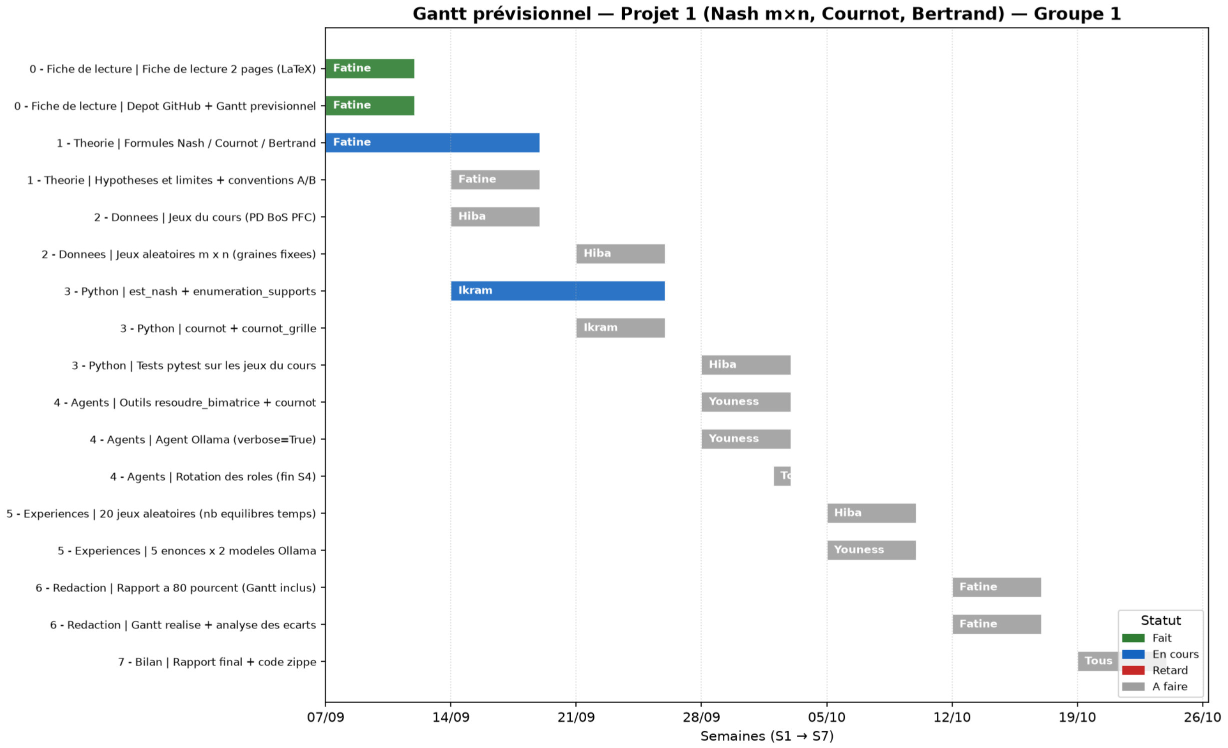

Figure 1 – Gantt prévisionnel du projet (S1 → S7), avec la rotation des rôles prévue en fin de S4.

- [Fatine] Fiche de lecture Phase 0 (formules + hypothèses)
- [Ikram] Solveur de Nash m×n : énumération des supports
- [Youness] Outils resoudre_bimatrice et cournot exposés à l'agent
- [Hiba] Tests pytest des jeux du cours + tests hors ligne